

1. Project Overview

1.1 Objective

In this lab project, you will extend the AXI-Lite interface (from Lab 3) to a full AXI bus protocol and integrate it with a DMA controller (from Lab 4) for high-throughput data transfers. The entire system must enforce **in-order transaction completion** to simplify design and verification.

1.2 Learning Outcomes

Upon completion of this project, you will be able to:

- Design and implement a full AXI bus interface with burst capability.
- Integrate a DMA controller as an AXI master for bulk data movement.
- Verify protocol compliance and system functionality through simulation.

1.3 Deliverables

Deliverable	Description	Weight
RTL Code	Verilog/SystemVerilog implementation of all modules	40%
Testbench	Simulation environment with directed and random tests	20%
Simulation Waveforms	Screenshots showing key transactions	15%
Design Report	Architecture description, FSM diagrams, and analysis	25%

2. System Architecture

2.1 Module Descriptions

Module	Description	Source
--------	-------------	--------

AXI_Full_Master	Full AXI master with burst support, in-order responses	Extend Lab 3
AXI_Full_Slave	Full AXI slave supporting burst reads/writes	Extend Lab 3
DMA_Controller	AXI master for bulk data transfers	Lab 4 (modify)
AXI_Interconnect	Connects masters and slaves (optional for Phase 2)	New

3. Project Phases & Milestones

Phase 1: Full AXI Implementation

3.1 AXI Full Interface Requirements

Parameter	Specification
Data width	32-bit
Address width	32-bit
Burst types supported	INCR only
Max burst length	16 beats
Transaction ordering	Strictly in-order
ID width	4 bits (simplified – fixed or single ID)

Response types	OKAY,SLVERR
----------------	-------------

Task 1.1: Extend AXI-Lite to Full AXI

1. Add burst support for **INCR** (incrementing) bursts.
2. Implement **5 independent channels**:
 - **Write Address (AW)**: with burst length (**AWLEN**)
 - **Write Data (W)**: with strobes (**WSTRB**)
 - **Write Response (B)**: completion status
 - **Read Address (AR)**: with burst length (**ARLEN**)
 - **Read Data (R)**: with last indicator (**RLAST**)
3. Enforce **in-order transaction completion** (simplified ID handling).

Task 1.2: Write a Testbench for Full AXI

- Generate bursts of different lengths (1 to 16 beats).
- Verify correct data alignment and burst types.
- Check that responses arrive in order.

Task 1.3: Simulate and Debug

- Run simulations and capture waveforms showing burst transactions.
- Ensure all AXI handshaking signals (**VALID/READY**) function correctly.

Phase 2: DMA Integration

3.2 DMA Controller Requirements

Parameter	Specification
AXI interface	Full AXI master (not slave)
Burst mode	INCR only (simplified)
Transfer size	8, 16, 32 bits (configurable)
Max transfer length	256 beats (configurable)

Status signals	Busy, Done, Error
Response handling	In-order (track with counter)

Task 2.1: Modify DMA from Lab 4

- Replace AXI-Lite interface with **full AXI master**.
- Add support for **burst transfers** (INCR mode recommended).
- Implement **in-order response tracking** – DMA should issue a transaction and wait for the response before proceeding or use a simple FIFO with ordered completion.

Task 2.2: DMA Configuration Registers

Add configuration registers (APB accessible) for:

Register	Function
Source Address	Starting read address
Destination Address	Starting write address
Transfer Length	Number of bytes/beats
Burst Size	2, 4, or 8 bytes
Control	Start/stop control
Status	Busy, done, error flags

Task 2.3: System-Level Integration

1. Connect the DMA to the full AXI interconnect.
2. Simulate a **memory-to-memory transfer** using DMA.
3. Verify that data is correctly read from source and written to destination.

4. Confirm **in-order completion** of DMA operations.

4. Submission Checklist

Item	Completed?
RTL code for all modules	
Testbench (Top Module)	
Waveform screenshots	
Design report (PDF)	